

Maximum Likelihood Estimation (MLE)

Table of Contents

1. [Introduction](#)
 2. [The Likelihood Function](#)
 3. [Maximum Likelihood Principle](#)
 4. [Log-Likelihood](#)
 5. [MLE for Common Distributions](#)
 6. [MLE via Optimization](#)
 7. [Properties of MLE](#)
 8. [Practical Examples](#)
-

Introduction

Maximum Likelihood Estimation (MLE) is a method to estimate parameters of a probability distribution by finding the parameter values that make the observed data most probable.

Core idea: "Which parameter values would have been most likely to generate the data we observed?"

Real-world example: Finding the submarine

- We have observations (sonar pings, debris locations, etc.)
 - We have a model with parameters (e.g., center location, spread, correlation)
 - MLE finds the parameters that best explain our observations
-

The Likelihood Function

Definition

The **likelihood function** $L(\theta; \text{data})$ is the probability of observing the data given parameter values θ .

For a single observation x :

$$\begin{aligned} L(\theta; x) &= P(X = x \mid \theta) \quad [\text{discrete}] \\ L(\theta; x) &= f(x; \theta) \quad [\text{continuous}] \end{aligned}$$

For multiple independent observations $x_1, x_2, \dots, x_n$:

$$\begin{aligned} L(\theta; x_1, \dots, x_n) &= \prod_i P(x_i \mid \theta) \quad [\text{discrete}] \\ L(\theta; x_1, \dots, x_n) &= \prod_i f(x_i; \theta) \quad [\text{continuous}] \end{aligned}$$

Likelihood vs Probability

Important distinction:

- **Probability:** Fix θ , vary $x \rightarrow P(X=x|\theta)$
 - "Given the parameters, how probable is this observation?"
- **Likelihood:** Fix x , vary $\theta \rightarrow L(\theta|x)$
 - "Given this observation, how likely are these parameters?"

Key point: Likelihood is NOT a probability distribution over θ !

- Doesn't have to integrate/sum to 1
- Can be > 1 for continuous distributions

Example: Coin Flips

Scenario: Flip a coin 10 times, get 7 heads.

Model: Binomial with parameter θ (probability of heads)

Likelihood:

$$L(\theta; 7 \text{ heads in } 10 \text{ flips}) = C(10,7) \theta^7 (1-\theta)^3$$

where $C(10,7) = 10!/(7!3!)$ is a constant.

```
def likelihood_coin(theta, n_heads=7, n_total=10):
    """Likelihood of  $\theta$  given observed coin flips"""
    from scipy.special import comb
    return comb(n_total, n_heads) * theta**n_heads * (1-theta)**(n_total-n_heads)

# Plot likelihood function
thetas = np.linspace(0, 1, 100)
likelihoods = [likelihood_coin(theta) for theta in thetas]

plt.plot(thetas, likelihoods)
plt.xlabel('θ (probability of heads)')
plt.ylabel('Likelihood')
plt.title('Likelihood Function')
plt.axvline(0.7, color='r', linestyle='--', label='θ=0.7 (MLE)')
plt.legend()
plt.show()
```

Maximum Likelihood Principle

Definition

The **Maximum Likelihood Estimate** (MLE) is the parameter value that maximizes the likelihood:

$$\hat{\theta}_{MLE} = \operatorname{argmax}_{\theta} L(\theta; \text{data})$$

In words: Find the θ that makes the observed data most likely.

Why It Makes Sense

Intuition: If we observed this data, the parameters that generated it are probably those that make this data highly probable!

Example: Coin flips

- Observed: 7 heads in 10 flips
- MLE: $\hat{\theta} = 7/10 = 0.7$
- Makes sense! Best guess is observed frequency

Finding the MLE

Method 1: Analytical (if possible)

1. Write down likelihood $L(\theta)$
2. Take derivative: $dL/d\theta$
3. Set to zero: $dL/d\theta = 0$
4. Solve for θ

Method 2: Optimization (numerical)

1. Define likelihood function
2. Use optimization algorithm
3. Find θ that maximizes $L(\theta)$

Log-Likelihood

Definition

The **log-likelihood** is the natural logarithm of the likelihood:

$$\ell(\theta) = \log L(\theta) = \log \prod_i f(x_i; \theta) = \sum_i \log f(x_i; \theta)$$

Why Use Log-Likelihood?

1. Products → Sums

$$L(\theta) = f(x_1) \cdot f(x_2) \cdot \dots \cdot f(x_n) \quad [\text{product, hard to compute}]$$

$$\ell(\theta) = \log f(x_1) + \log f(x_2) + \dots + \log f(x_n) \quad [\text{sum, easy!}]$$

2. Numerical Stability

- Probabilities can be tiny (e.g., 10^{-300})
- Products underflow to 0
- Logs keep numbers in reasonable range

3. Same Maximum

- log is monotonic increasing
- $\operatorname{argmax} L(\theta) = \operatorname{argmax} \log L(\theta)$

4. Easier Math

- Derivatives are simpler
- Sums are easier than products

Example: Normal Distribution

Likelihood for $X \sim N(\mu, \sigma^2)$:

$$L(\mu, \sigma^2; x_1, \dots, x_n) = \prod_i (1/\sqrt{2\pi\sigma^2}) \exp(-(x_i - \mu)^2/(2\sigma^2))$$

Log-likelihood:

$$\begin{aligned}
 \ell(\mu, \sigma^2) &= \sum_i \log\left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)\right] \\
 &= \sum_i \left[-\frac{1}{2}\log(2\pi\sigma^2) - \frac{(x_i - \mu)^2}{2\sigma^2}\right] \\
 &= -n/2 \log(2\pi\sigma^2) - 1/(2\sigma^2) \sum_i (x_i - \mu)^2
 \end{aligned}$$

Much cleaner!

MLE for Common Distributions

1. Bernoulli / Binomial

Model: $X \sim \text{Bernoulli}(\theta)$, n trials, k successes

MLE:

$$\hat{\theta} = k/n$$

Example: 7 heads in 10 flips $\rightarrow \theta = 0.7$

```

n_success = 7
n_total = 10
theta_mle = n_success / n_total
print(f"MLE:  $\hat{\theta} = \{theta\_mle\}$ ")

```

2. Normal Distribution (Known Variance)

Model: $X \sim N(\mu, \sigma^2)$, σ^2 known

MLE:

$$\hat{\mu} = (1/n) \sum_i x_i = \bar{x} \quad (\text{sample mean})$$

Example:

```
data = np.array([1.2, 1.5, 1.3, 1.7, 1.4])
mu_mle = np.mean(data)
print(f"MLE:  $\mu^{\hat{}} = \{\text{mu\_mle}\}$ ")
```

3. Normal Distribution (Both Parameters Unknown)

Model: $X \sim N(\mu, \sigma^2)$ **MLE:**

$$\mu^{\hat{}} = (1/n) \sum_i x_i \quad (\text{sample mean})$$

$$\sigma^2 = (1/n) \sum_i (x_i - \mu)^2 \quad (\text{sample variance, biased!})$$
Note: This is the *biased* estimator. Unbiased uses (n-1) instead of n.

```
data = np.array([1.2, 1.5, 1.3, 1.7, 1.4])
mu_mle = np.mean(data)
sigma2_mle = np.mean((data - mu_mle)**2) # Biased MLE
sigma2_unbiased = np.var(data, ddof=1) # Unbiased estimator

print(f"MLE:  $\mu^{\hat{}} = \{\text{mu\_mle}\}$ ,  $\sigma^2 = \{\text{sigma2\_mle}\}$ ")
print(f"Unbiased:  $\sigma^2 = \{\text{sigma2\_unbiased}\}$ ")
```

4. Multivariate Normal

Model: $X \sim N(\mu, \Sigma)$ **MLE:**

$$\mu^{\hat{}} = (1/n) \sum_i x_i$$

$$\hat{\Sigma} = (1/n) \sum_i (x_i - \mu^{\hat{}})(x_i - \mu^{\hat{}})^T$$

```
samples = np.random.multivariate_normal([5, 5], [[1, 0.5], [0.5, 1]], 100)

mu_mle = np.mean(samples, axis=0)
# Note: np.cov uses (n-1), which is unbiased. For MLE (biased), use:
Sigma_mle = np.cov(samples.T, bias=True) # bias=True gives MLE

print(f"MLE  $\mu$ : \{mu_mle\}")
print(f"MLE  $\hat{\Sigma}$ : \{Sigma_mle\}")
```

MLE via Optimization

When analytical solutions don't exist, use numerical optimization.

Negative Log-Likelihood

Convention: Optimizers minimize, so we minimize **negative** log-likelihood:

$$\hat{\theta} = \operatorname{argmin}_{\theta} [-\ell(\theta)] = \operatorname{argmax}_{\theta} \ell(\theta)$$

Example: Custom Distribution

Scenario: Submarine location modeled by a "cross distribution" with parameters (ctr_x, ctr_y, span, leg, angle).

Steps:

1. Define log-likelihood function
2. Define negative log-likelihood
3. Optimize using numerical methods


```
def negative_log_likelihood(theta, observations, model_class):
    """
    Compute negative log-likelihood

    Parameters:
    -----
    theta : array
        Parameters [ctr_x, ctr_y, span, leg, angle]
    observations : array (n, 2)
        Observed data points
    model_class : class
        Distribution class (e.g., CrossDistribution)

    Returns:
    -----
    neg_llik : float
        Negative log-likelihood
    """
    # Create model with these parameters
    ctr = theta[0:2]
    span = theta[2]
    leg = theta[3]
    angle = theta[4]

    model = model_class(ctr=ctr, span=span, leg=leg, angle=angle)

    # Compute log-likelihood of each observation
    log_likelihoods = np.array([model.llik(obs) for obs in observations])

    # Sum to get total log-likelihood
    total_llik = np.sum(log_likelihoods)

    # Return negative (for minimization)
    return -total_llik
```

Optimization Approaches

We'll cover this in detail in the next guide, but briefly:

1. Gradient-based methods (if you can compute gradients)

- Gradient descent
- Newton's method
- BFGS

2. Gradient-free methods (if you can't compute gradients)

- Nelder-Mead simplex
- Powell's method
- **Stochastic hill climbing** (random search with acceptance criterion)

Properties of MLE

1. Consistency

As $n \rightarrow \infty$, $\hat{\theta}_{MLE} \rightarrow \theta_{true}$ (converges to true parameter)

2. Asymptotic Normality

For large n :

$$\hat{\theta}_{MLE} \sim N(\theta_{true}, I(\theta)^{-1}/n)$$

where $I(\theta)$ is the Fisher information.

Implication: MLE is approximately normally distributed around the true value.

3. Efficiency

MLE has the smallest asymptotic variance among all consistent estimators (Cramér-Rao bound).

Implication: MLE is "best" in a certain sense.

4. Invariance

If $\hat{\theta}$ is MLE of θ , then $g(\hat{\theta})$ is MLE of $g(\theta)$ for any function g .

Example:

- $\hat{\theta}$ is MLE of μ
- $\hat{\theta}^2$ is MLE of μ^2
- $\log(\hat{\theta})$ is MLE of $\log(\mu)$

5. Not Always Unbiased!

MLE can be biased for finite samples.

Example: Variance estimator

- MLE: $\sigma^2 = (1/n) \sum (x_i - \hat{\mu})^2$ (biased)
 - Unbiased: $s^2 = (1/(n-1)) \sum (x_i - \hat{\mu})^2$
-

Practical Examples

Example 1: Simple Normal Distribution

```
# Generate data
np.random.seed(42)
true_mu = 5.0
true_sigma = 2.0
data = np.random.normal(true_mu, true_sigma, 100)

# MLE estimates
mu_mle = np.mean(data)
sigma_mle = np.std(data) # Note: uses n, not n-1

print(f"True parameters:  $\mu$ ={true_mu},  $\sigma$ ={true_sigma}")
print(f"MLE estimates:  $\hat{\mu}$ ={mu_mle:.3f},  $\hat{\sigma}$ ={sigma_mle:.3f}")

# Visualize
from scipy.stats import norm

x = np.linspace(0, 10, 100)
plt.hist(data, bins=20, density=True, alpha=0.5, label='Data')
plt.plot(x, norm.pdf(x, true_mu, true_sigma), 'r-', label='True distribution')
plt.plot(x, norm.pdf(x, mu_mle, sigma_mle), 'b--', label='MLE distribution')
plt.legend()
plt.title('MLE for Normal Distribution')
plt.show()
```

Example 2: Comparing Sequences

Scenario: We have a model (submarine_pmf) and several observation sequences. Which sequence is most compatible with the model?

```

def log_likelihood_sequence(sequence, pmf):
    """
    Compute log-likelihood of a sequence of grid positions

    Parameters:
    -----
    sequence : array (n, 2)
        Sequence of [x, y] positions
    pmf : array (16, 16)
        Probability mass function

    Returns:
    -----
    llik : float
        Total log-likelihood
    """
    llik = 0.0
    for x, y in sequence:
        x, y = int(x), int(y)
        if pmf[x, y] > 0:
            llik += np.log(pmf[x, y])
        else:
            llik += -np.inf # Probability 0 → log-likelihood -∞

    return llik

# Compute for each sequence
from submarine import submarine_samples, submarine_pmf

lliks = {}
for name, sequence in submarine_samples.items():
    llik = log_likelihood_sequence(sequence, submarine_pmf)
    lliks[name] = llik
    print(f"{name}: log-likelihood = {llik:.2f}")

# Find most likely
most_likely = max(lliks, key=lliks.get)
print(f"\nMost likely sequence: {most_likely}")

```

Example 3: Fitting Multivariate Normal

```
# Load observations
observations = np.loadtxt('data/submarine.txt')

# MLE for multivariate normal
mu_mle = np.mean(observations, axis=0)
Sigma_mle = np.cov(observations.T)

print("MLE estimates:")
print(f" $\hat{\mu} = \{\text{mu\_mle}\}$ ")
print(f" $\hat{\Sigma} = \{\text{Sigma\_mle}\}$ ")

# Compute log-likelihood
from scipy.stats import multivariate_normal

mvn = multivariate_normal(mu_mle, Sigma_mle)
log_likelihood = np.sum(mvn.logpdf(observations))
print(f"\nLog-likelihood: {log_likelihood:.2f}")

# Compare to samples from fitted model
samples_fitted = mvn.rvs(len(observations))

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.scatter(observations[:, 0], observations[:, 1], alpha=0.5)
plt.title('Observations')
plt.xlabel('X')
plt.ylabel('Y')

plt.subplot(1, 2, 2)
plt.scatter(samples_fitted[:, 0], samples_fitted[:, 1],
            alpha=0.5, color='red')
plt.title('Samples from MLE Model')
plt.xlabel('X')
plt.ylabel('Y')

plt.tight_layout()
plt.show()
```

Example 4: Model Selection

Scenario: Which model fits the data better?

```
# Model 1: Full covariance
mu1 = np.mean(observations, axis=0)
Sigma1 = np.cov(observations.T)
mvn1 = multivariate_normal(mu1, Sigma1)
llik1 = np.sum(mvn1.logpdf(observations))

# Model 2: Diagonal covariance (independence)
mu2 = np.mean(observations, axis=0)
Sigma2 = np.diag(np.var(observations, axis=0))
mvn2 = multivariate_normal(mu2, Sigma2)
llik2 = np.sum(mvn2.logpdf(observations))

print(f"Model 1 (full): log-likelihood = {llik1:.2f}")
print(f"Model 2 (diagonal): log-likelihood = {llik2:.2f}")
print(f"Difference: {llik1 - llik2:.2f}")

if llik1 > llik2:
    print("Model 1 fits better (correlation matters!)")
else:
    print("Model 2 fits better (independence assumption OK)")
```

Common Mistakes

Mistake 1: Confusing Likelihood and Probability

```
# Likelihood is NOT a probability!
# It doesn't have to sum/integrate to 1

theta_values = np.linspace(0, 1, 100)
likelihoods = [likelihood_coin(theta) for theta in theta_values]
print(f"Sum of likelihoods: {np.sum(likelihoods)}") # NOT 1!
```

Mistake 2: Not Using Log-Likelihood

```
# BAD: Multiplying tiny probabilities
L = 1.0
for x in observations:
    L *= pmf[int(x[0]), int(x[1])] # Underflows to 0!

# GOOD: Summing log-probabilities
llik = 0.0
for x in observations:
    llik += np.log(pmf[int(x[0]), int(x[1])]) # Stable!
```

Mistake 3: Forgetting Negative for Optimization

```
# WRONG: Minimizing log-likelihood (finds worst fit!)
result = minimize(log_likelihood, initial_guess)

# CORRECT: Minimizing NEGATIVE log-likelihood
result = minimize(lambda theta: -log_likelihood(theta), initial_guess)
```


Mistake 4: Not Checking Constraints

```
# Some parameters have constraints!
# E.g., variance must be positive, probabilities in [0,1]

def negative_log_likelihood_constrained(theta):
    # Extract parameters
    mu, sigma = theta

    # Check constraints
    if sigma <= 0:
        return np.inf # Invalid!

    # Compute negative log-likelihood
    ...
```

Summary

Maximum Likelihood Estimation:

$$\hat{\theta}_{MLE} = \operatorname{argmax}_{\theta} L(\theta; \text{data}) = \operatorname{argmax}_{\theta} \prod_i f(x_i; \theta)$$

Log-Likelihood (easier to work with):

$$\begin{aligned} \ell(\theta) &= \log L(\theta) = \sum_i \log f(x_i; \theta) \\ \hat{\theta}_{MLE} &= \operatorname{argmax}_{\theta} \ell(\theta) \end{aligned}$$

For Optimization (minimizers):

$$\hat{\theta}_{MLE} = \operatorname{argmin}_{\theta} [-\ell(\theta)]$$

Common MLEs:

- Bernoulli: $\hat{\theta} = k/n$ (proportion of successes)
- Normal: $\hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = (1/n)\sum (x_i - \hat{\mu})^2$
- Multivariate Normal: $\hat{\mu} = \bar{x}$, $\hat{\Sigma} = (1/n)\sum (x_i - \hat{\mu})(x_i - \hat{\mu})^T$

Properties:

- Consistent ($\hat{\theta} \rightarrow \theta$ as $n \rightarrow \infty$)
- Asymptotically normal
- Efficient (smallest variance)
- Invariant under transformations
- Not necessarily unbiased!

Practical Steps:

1. Write down likelihood $L(\theta)$ or log-likelihood $\ell(\theta)$
2. If possible, find analytical solution (set derivative to 0)
3. Otherwise, use numerical optimization
4. Always work with log-likelihood for numerical stability
5. Verify constraints are satisfied

Next: How to do the numerical optimization (stochastic hill climbing, etc.)